Designated-initializers for Base Classes

Document #:	P2287R3 [Latest] [Status]
Date:	2024-09-09
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

[P2287R2] proposed only allowing indirectly initializing members using the designated syntax. R3 additionally allows directly initializing base class subobjects *without* a designator. Also adds an implementation.

[P2287R1] proposed a novel way of naming base classes, as well as a way for naming indirect non-static data members. This revision *only* supports naming direct or indirect non-static data members, with no mechanism to name base classes. Basically only what Matthias suggested.

[P2287R0] proposed a single syntax for a *designated-initializer* that identifies a base class. Based on a reflector suggestion from Matthias Stearn, this revision extends the syntax to allow the brace-elision version of *designated-initializer*: allow naming indirect non-static data members as well. Also actually correctly targeting EWG this time.

§ 2 Introduction

[P0017R1] extended aggregates to allow an aggregate to have a base class. [P0329R4] gave us designated initializers, which allow for much more expressive and functional initialization of aggregates. However, the two do not mix: a designated initializer can currently only refer to a direct non-static data members. This means that if I have a type like:

```
struct A {
    int a;
};

struct B : A {
    int b;
};
```

While I can initialize an `A` like `A{.a=1}`, I cannot designated-initialize `B`. An attempt like `B{{.a=1}, .b=2}` runs afoul of the rule that the initializers must either be all designated or none designated. But there is currently no way to designate the base class here.

Which means that my only options for initializing a `B` are to fall-back to regular aggregate initialization and write either `B{{1}, 2}` or `B{1, 2}`. Neither are especially satisfactory.

§ 2.1 Design Space

There are basically three potential approaches for being able to designated-initialize `B`:

```
// provide a way to name the base class
auto b1 = B{.A={.a=1}, .b=2};

// allow mixing designated and non-designated
auto b2 = B{{.a=3}, .b=4};

// allow directly initializing indirect members
auto b3 = B{.a=5, .b=6};
```

A previous revision of this paper proposed allowing only `b1` — coming up with a way to name the base class. This is much more complicated than naming an aggregate member because base classes aren't just *`identifier`*s, they can include template parameters. This revision eschews that approach entirely.

This leaves the other two options. When this paper was discussed in Varna, there was some desire expressed for having braces around the base class subobject (as in `b2` above). While allowing for optional braces is straightforward enough (and is simply a matter of refining the restriction on mixing designated and non-designated initializers), I would be strongly opposed to mandating braces.

Designated-initialization mirrors assignment. The goal is to do this substitution:

Assignment	Initialization

`[&]{`
`  B b;`
`  b.a = 5;`
`  b.b = 6;`
`  return b;`
`}();`
	`B{.a=5, .b=6}`

We can directly assign into `a`, so we should be able to directly initialize into it. Of course, in C the two forms are identical (assuming C had lambdas) while in C++ they aren't necessarily due to the existence of constructors. This makes designated initialization in C a much simpler problem. Plus C doesn't have base classes. But the essence should be the same — mandating braces breaks that.

Additionally, in many cases, the point of inheritance of aggregates is not being we actually need an "is-a" relationship but rather to compose aggregate members. I don't need `B` to be an `A`, I need `B` to have the same members as `A` with the same convenient access syntax — as opposed to having a member of type `A` and needing an extra accessor in the chain. Forcing the user to initialize `B` with braces forces the user to be aware of what is basically an implementation detail. It's frustrating enough to deal with the requirement that initializers be in-order, additionally requiring braces is too much.

Lastly, aggregate initialize *already* allows for brace elision. `B{1, 2}` is well-formed today, despite the `1` initializing a base class subobject and `2` initializing a direct member. So it strikes me as especially contrary to the design to suddenly mandate braces here.

But *allowing* (not mandating) braces? That seems totally fine. Also, notably, gcc in `-std=c++17` mode — still to this day on trunk — supports `B{{.a=1}, .b=2}`. We actually had some code break while upgrading to C++20 that initialized aggregates in this way. While I think `B{.a=1, .b=2}` is the clearest way to initialize this type, `B{{.a=1}, .b=2}` is still great — and both are substantial improvements over the best you can do in valid C++20 today, which would be either `B{{.a=1}, 2}` or simply `B{1, 2}`.

§ 3 Proposal

This paper proposes extending designated initialization syntax to:

1. allow directly initializing base class aggregate elements, and
2. allow mixing designated and non-designated initializers, if all of the non-designated initializers are at the beginning of the *initializer-list* and are used to initialize base class subobjects.

In short, based on the above declarations of `A` and `B`, this proposal allows all of the following declarations:

```
B{{1}, 2}          // already valid in C++17
B{1, 2}            // already valid in C++17

B{.a=1, .b=2}      // proposed
B{{.a=1}, .b=2}    // proposed
```

```
B{.a{1}, .b{2}}   // proposed
B{.b=2, .a=1}     // still ill-formed
```

§ 3.1 Naming the base classes

The original revisions of this paper dealt with how to name the `A` base class of `B`, and what this means for more complicated base classes (such at those with template parameters). This revision eschews that approach entirely: it's simpler to just stick with naming members, direct and indirect. After all, that's how these aggregates will be interacted with.

What this means is that while this paper proposes that this works:

```
struct A { int a; };
struct B : A { int b; };

auto b = B{.a=1, .b=2};
```

And in a type that has a base class that is not an aggregate, you can use the mix-and-match form:

```
struct C : std::string { int c; };

auto c1 = C{"hello", .c=3};
auto c2 = C{{"hello"}, .c=3};
```

If you have a hierarchy with repeated members, you'll likewise have to use the mix-and-match form:

```
struct D { int x; };
struct E : D { int x; };

auto e1 = E{.x=1};         // initializes E::x, not D::x
auto e2 = E{{.x=1}, .x=2}; // initializes D::x to 1 and E::x to 2
auto e3 = E{D{1}, .x=2};   // likewise
```

Coming up with a way to *name* the base class subobject of a class seems useful, but that's largely orthogonal. It can be done later.

§ 3.2 Naming all the subobjects

The current wording we have says that, from 9.4.2 [dcl.init.aggr]/3.1:

(3.1) If the initializer list is a brace-enclosed *designated-initializer-list*, the aggregate shall be of class type, the *identifier* in each designator shall name a direct non-static data member of the class […]

And, from 9.4.5 [dcl.init.list]/3.1 (conveniently, it's 3.1 in both cases):

> (3.1) If the *braced-init-list* contains a *designated-initializer-list*, `T` shall be an aggregate class. The ordered identifiers in the designators of the *designated-initializer-list* shall form a subsequence of the ordered identifiers in the direct non-static data members of `T`. Aggregate initialization is performed ([dcl.init.aggr]).

The proposal here is to extend both of these rules to cover not just the direct non-static data members of `T` but also all indirect members, such that every interleaving base class is also an aggregate class. That is:

```
struct A { int a; };
struct B : A { int b; };
struct C : A { C(); int c; };
struct D : C { int d; };

A{.a=1};       // okay since C++17
B{.a=1, .b=2}; // proposed okay, 'a' is a direct member of an aggregate
        class
               // and A is a direct base
C{.c=1};       // error: C is not an aggregate
D{.a=1};       // error: 'a' is a direct member of an aggregate class
               // but an intermediate base class (C) is not an aggregate
```

Or, put differently, every identifier shall name a non-static data member that is not a (direct or indirect) member of any base class that is not an aggregate.

Also this is still based on lookup rules, so if the same name appears in multiple base classes, then either it's only the most derived one that counts:

```
struct X { int x; };
struct Y : X { int x; };
Y{.x=1}; // initializes Y::x
```

or is ambiguous:

```
struct X { int x; };
struct Y { int x; };
struct Z : X, Y { };
Z{.x=1}; // error:: ambiguous which X
```

would be ill-formed on the basis that `Z::x` is ambiguous.

§ 3.3 Impact on Existing Code

There is one case I can think of where code would change meaning:

```
struct A { int a; };
struct B : A { int b; };

void f(A); // #1
void f(B); // #2

void g() {
    f({.a=1});
}
```

In C++23, `f({.a=1})` calls `#1`, as it's the only viable candidate. But with this change, `#2` also becomes a viable candidate, so this call becomes ambiguous. I have no idea how much such code exists. This is, at least, easy to fix.

I don't think there's a case where code would change from one valid meaning to a different valid meaning - just from valid to ambiguous.

§ 3.4 Implementation Experience

I implemented this in clang in a very literal way — by synthesizing a new designated-initializer-list to initialize the base classes in the situations where that comes up. There is probably a more straightforward way to do this, but all the examples in the paper work.

§ 4 Wording

§ 4.1 Strategy

The wording strategy here is as follows. Let's say we have this simple case:

```
struct A { int a; };
struct B : A { int b; };
```

In the current wording, `B` has two elements (the `A` direct base class and then the `b` member). The initialization `B{.b=2}` is considered to have one explicitly initialized element (the `b`, initialized with `2`) and then the `A` is not explicitly initialized and cannot have a default member initializer, so it is copy-initialized from `{}`.

The strategy to handle `B{.a=1, .b=2}` is to group the indirect non-static data members under their corresponding direct base class and to treat those base class elements as being explicitly initialized. So here, the `A` element is explicitly initialized from `{.a=1}` and the `b` element continues to be explicitly initialized from `2`. And then this applies recursively, so given:

```
struct C : B { int c; };
```

With `C{.a=1, .c=2}`, we have:

— the `B` element is explicitly initialized from `{.a=1}`, which leads to:
  — the `A` element is explicitly initialized from `{.a=1}`, which leads to:
    — the `a` element is explicitly initialized from `1`
  — the `b` element is not explicitly initialized and has no default member initializer, so it is copy-initialized from `{}`
— the `c` element is explicitly initialized from `2`

§ 4.2 Actual Wording

Change the grammar in 9.4.1 [dcl.init.general]/1 to allow a *designated-initializer-list* to start with an *initializer-list*:

```
  designated-initializer-list:
+   designated-only-initializer-list
+   initializer-list , designated-only-initializer-list

+ designated-only-initializer-list:
    designated-initializer-clause
-   designated-initializer-list , designated-initializer-clause
+   designated-only-initializer-list , designated-initializer-clause
```

Add a new term after we define what an aggregate and the elements of an aggregate are:

* The *designatable members* of an aggregate `T` are:

(*.1) — For each direct base class `C` of `T` that is an aggregate class, the designatable members of `C` for which lookup for that member in `T` finds the member of `C`,

(*.2) — The ordered *identifiers* in the direct non-static members of `T`.

Extend 9.4.2 [dcl.init.aggr]/3.1:

(3.1) If the initializer list is a brace-enclosed *designated-initializer-list*, the aggregate shall be of class type~~, the identifier in each designator shall name a direct non-static data member of the class, and the explicitly initialized elements of the aggregate are the elements that are, or contain, those members.~~ `C` .

(3.1.1) — If the initializer list contains a *initializer-list*, then each *initializer-clause* in the *initializer-list* shall appertain (see below) to a base class subobject of `C` and the *identifier* in each *designator* shall name a direct non-static data member of `C`, and the explicitly initialized elements of the aggregate are those base class subobjects and direct members.

(3.1.2) — Otherwise, the *identifier* in each *designator* shall name a designatable member of the class. The explicitly initialized elements of the aggregate are the elements that are, or contain (in the case of an anonymous union or of a base class) those members.

And extend 9.4.2 [dcl.init.aggr]/4 to cover base class elements:

4 For each explicitly initialized element:

(4.0) — If the initializer list is a brace-enclosed *designated-initializer-list* with no *initializer-list* and the element is a direct base class, then let `C` denote that direct base class and let `T` denote the class. The element is initialized from a synthesized brace-enclosed *designated-initializer-list* containing each *designator* for which lookup in `T` names a direct or indirect non-static data member of `C` in the same order as in the original *designated-initializer-list*.

[*Example 1:*

```
struct A { int a; };
struct B : A { int b; };
struct C : B { int c; };

// the A element is intialized from {.a=1}
B x = B{.a=1};

// the B element is initialized from {.a=2, .b=3}
// which leads to its A element being initialized from {.a=2}
C y = C{.a=2, .b=3, .c=4};

struct A2 : A { int a; };

// the A element is not explicitly initialized
A2 z = {.a=1};
```

— *end example*]

(4.1) — ~~If~~ Otherwise, if the element is an anonymous union […]

(4.2) — Otherwise, if the initializer list is a brace-enclosed *designated-initializer-list* and the element is not a base class subobject, the element is initialized with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause* […]

(4.3) — Otherwise, either the initializer list is a brace-enclosed *initializer-list* or the element is a base class subobject and the initializer list is a brace-enclosed *designated-initializer-list* which contains an *initializer-list*. […]

[*Example 2:*

```
struct base1 { int b1, b2 = 42; };
struct base2 {
  base2() {
    b3 = 42;
  }
```

```
  int b3;
};
struct derived : base1, base2 {
  int d;
};

derived d1{{1, 2}, {}, 4};
derived d2{{}, {}, 4};
+ derived d3{{1, 2}, {}, .d=4};
```

initializes:

— `d1.b1` with `1`, `d1.b2` with `2`, `d1.b3` with `42`, `d1.d` with `4`, ~~and~~

— `d2.b1` with `0`, `d2.b2` with `42`, `d2.b3` with `42`, `d2.d` with `4`, and

— `d3.b1` with `1`, `d3.b2` with `2`, `d3.b3` with `42`, `d3.d` with `4`.

— *end example*]

Extend 9.4.5 [dcl.init.list]/3.1:

(3.1) If the *braced-init-list* contains a *designated-initializer-list*, `T` shall be an aggregate class. The ordered *identifier*s in the designators of the *designated-initializer-list* shall form a subsequence of designatable members [(dcl.init.aggr)] of `T`.

Aggregate initialization is performed ([dcl.init.aggr]).

[*Example 3:*

```
struct A { int x; int y; int z; };
A a{.y = 2, .x = 1};                // error: designator order does
    not match declaration order
A b{.x = 1, .z = 2};               // OK, b.y initialized to 0

+   struct B : A { int q; };
+   B e{.x = 1, .q = 3};                // OK, e.y and e.z initialized
        to 0
+   B f{.q = 3, .x = 1};               // error: designator order does
        not match declaration order

+   struct C { int p; int x; };
+   struct D : A, C { };
+   D g{.y=1, .p=2};                    // OK
+    D  h{.x=2};                              // error: x is not a
        designatable member

+   struct NonAggr { int na; NonAggr(int); };
+   struct E : NonAggr { int e; };
+    E  i{.na=1, .e=2};                        // error: na is not a
        designatable member
```

— *end example*]

Add an Annex C entry:

Affected sublcause: [dcl.init]
Change: Support for designated initialization of base classes of aggregates.
Rationale: New functionality.
Effect on original feature: Some valid C++23 code may fail to compile. For example:

```
struct A { int a; };
struct B : A { int b; };

void f(A); // #1
void f(B); // #2

void g() {
    f({.a=1}); // OK (calls #1) in C++23, now ill-formed (ambiguous)
}
```

§ 4.3 Feature-test Macro

Bump `__cpp_designated_initializers` in 15.11 [cpp.predefined]:

```
- __cpp_designated_initializers 201707L
+ __cpp_designated_initializers 2024XXL
```

§ 5 Acknowledgements

Thanks to Matthias Stearn for, basically, the proposal. Thanks to Tim Song for helping with design questions and wording.

§ 6 References

[P0017R1] Oleg Smolsky. 2015-10-24. Extension to aggregate initialization.
https://wg21.link/p0017r1
[P0329R4] Tim Shen, Richard Smith. 2017-07-12. Designated Initialization Wording.
https://wg21.link/p0329r4
[P2287R0] Barry Revzin. 2021-01-21. Designated-initializers for base classes.
https://wg21.link/p2287r0
[P2287R1] Barry Revzin. 2021-02-15. Designated-initializers for base classes.
https://wg21.link/p2287r1
[P2287R2] Barry Revzin. 2023-03-12. Designated-initializers for base classes.
https://wg21.link/p2287r2